# Lab 50: 14 files, shallow comments: split the review

Topic 4: Prompt Engineering & Structured Output · Task 4.6: Design multi-instance and multi-pass review architectures

Runs on Simulator 5: Claude Code for Continuous Integration · Run in class

Source: Official exam guide, sample question 12

## The question

A pull request modifies 14 files across the stock tracking module. Your single-pass review analyzing all files together produces inconsistent results: detailed feedback for some files but superficial comments for others, obvious bugs missed, and contradictory feedback—flagging a pattern as problematic in one file while approving identical code elsewhere in the same PR. How should you restructure the review?

**A.** Split into focused passes: analyze each file individually for local issues, then run a separate integration-focused pass examining cross-file data flow.

**B.** Require developers to split large PRs into smaller submissions of 3-4 files before the automated review runs.

**C.** Switch to a higher-tier model with a larger context window to give all 14 files adequate attention in one pass.

**D.** Run three independent review passes on the full PR and only flag issues that appear in at least two of the three runs.

## Simulate it live

```
# In the browser: open Simulator 5, then Lab 50
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 50

# In a terminal, from simulators/simulator-5-ci-cd/
uv run ci_review.py; echo "exit $?"
```

## What you should see

Only the integration pass finds the jobs.py call with the old signature.

Open `simulators/simulator-5-ci-cd/evidence/` and find it yourself.

## The answer

**A.** Per-file local passes, then one cross-file integration pass.

Splitting reviews into focused passes directly addresses the root cause: attention dilution when processing many files at once. File-by-file analysis ensures consistent depth, while a separate integration pass catches cross-file issues. Option B shifts burden to developers without improving the system. Option C misunderstands that larger context windows don't solve attention quality issues. Option D would actually suppress detection of real bugs by requiring consensus on issues that may only be caught intermittently.

## Now change one thing

Fix the loop in repo/stock/totals.py and the caller in repo/stock/jobs.py, rerun, and watch the blocking count fall.